

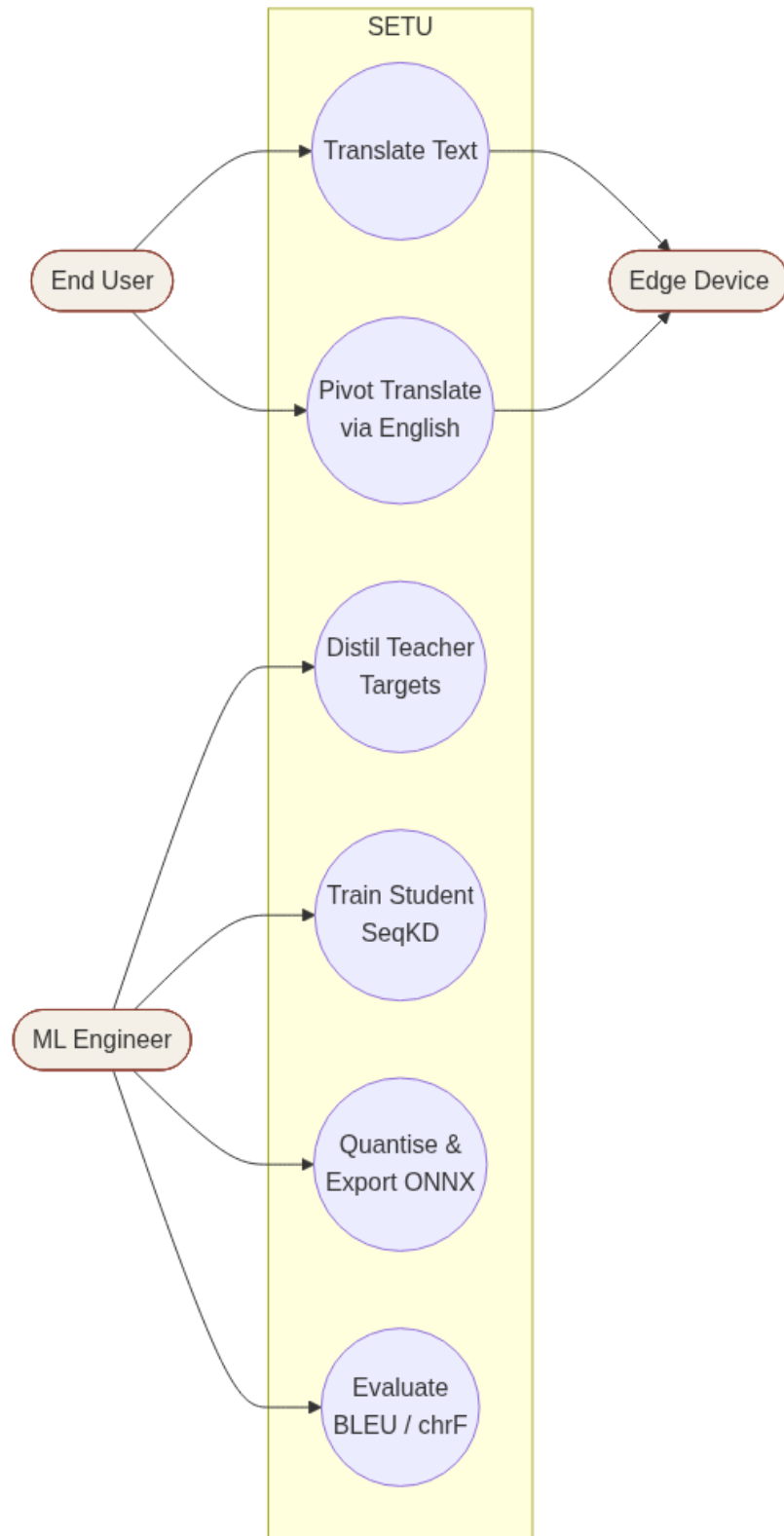


Figure 3.1: Use case diagram: End User, ML Engineer and Edge Device actors

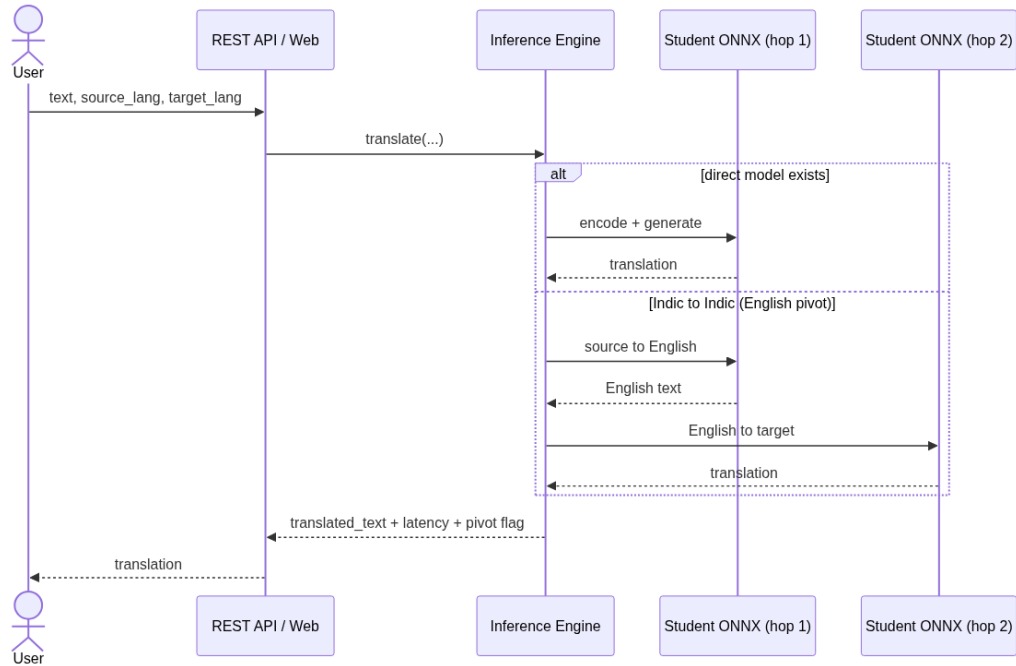


Figure 3.2: Sequence diagram: direct translation and the two-hop English pivot

3.2.3 Activity Diagram

Figure 3.3 shows the full training and deployment activity flow. Everything above the dividing line executes once per direction on a GPU; everything below executes on the user's device. The two decision points are the quality gate (does the student reach 80 % of teacher BLEU) and the routing decision at inference time.

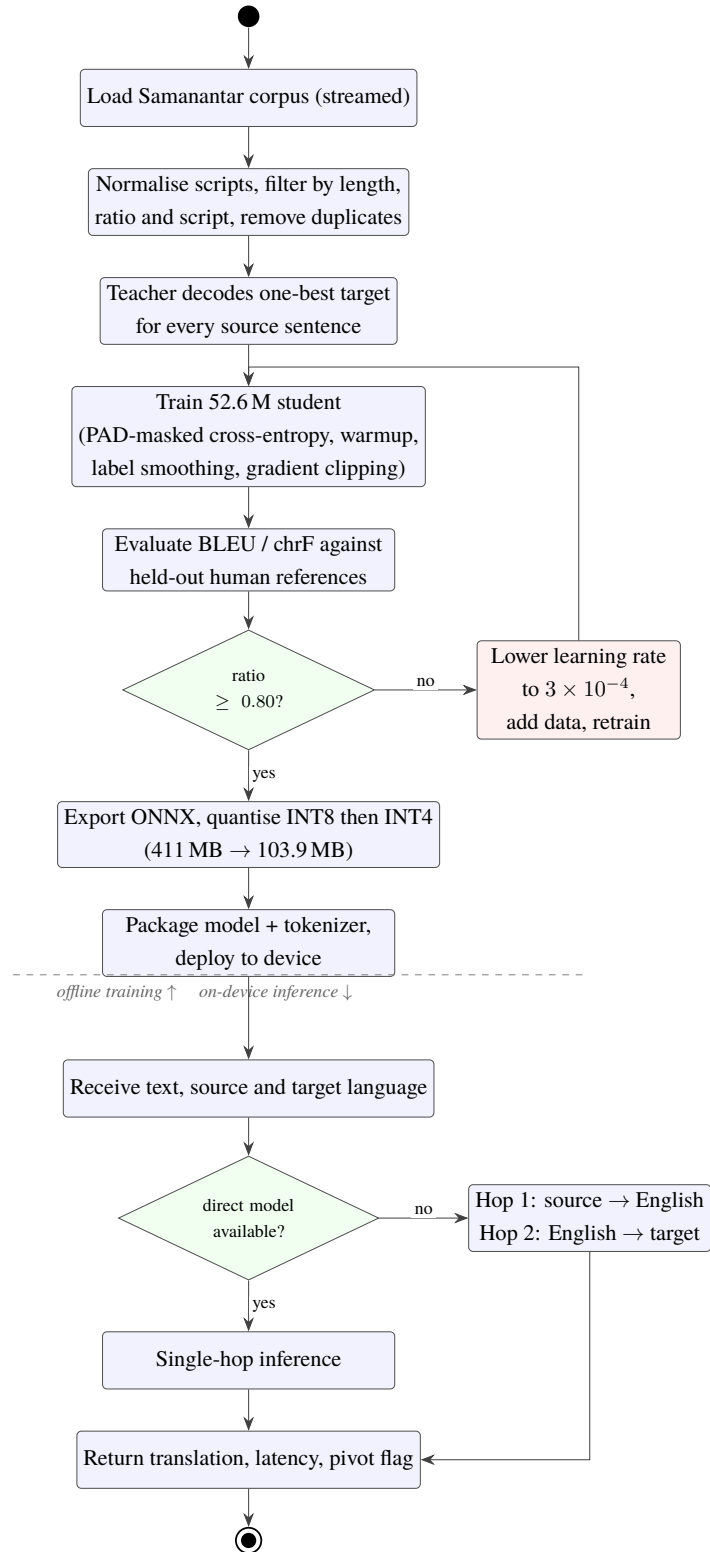


Figure 3.3: Activity diagram: offline training pipeline (above) and on-device inference (below)

3.2.4 State Chart Diagram

A model in SETU passes through a well-defined sequence of states. Figure 3.4 records them, including the two transitions that were exercised in practice: the collapse detected at evaluation and the recovery achieved by retraining at a lower learning rate on the same distilled corpus.

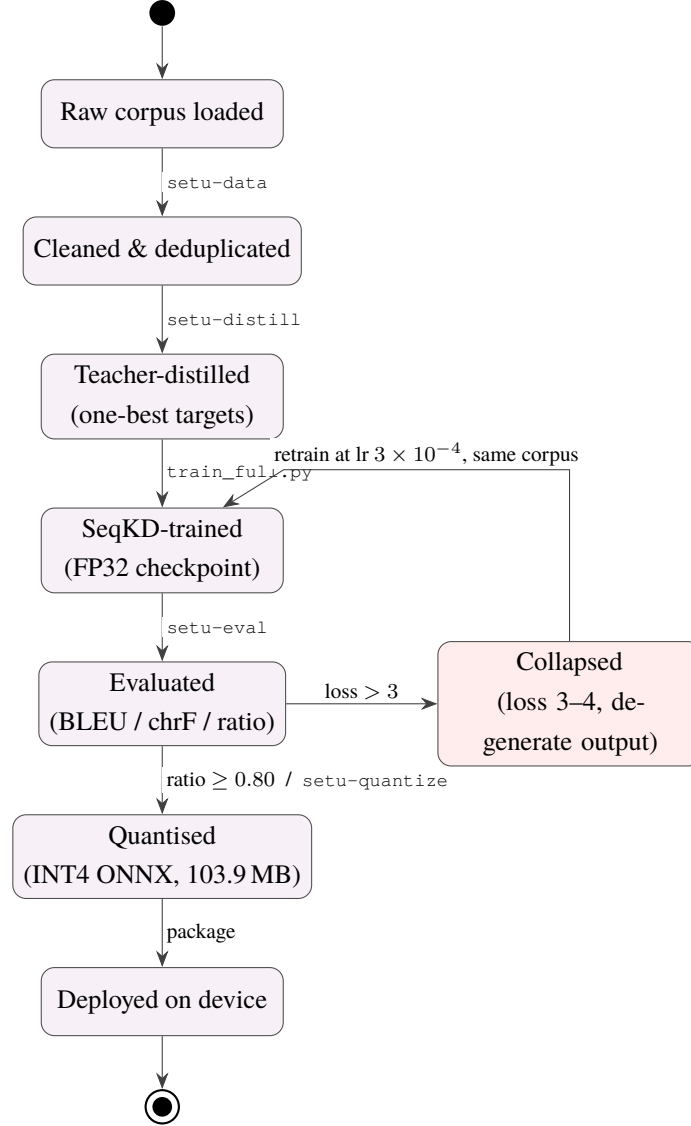


Figure 3.4: State chart diagram: model lifecycle from raw corpus to deployed artifact, including the observed collapse and recovery transitions

3.2.5 Data Flow Diagram

Figure 3.5 gives the level-1 data flow. The corpus is an external data source; the teacher is a transformation that produces the distilled data store; the trainer consumes that store and produces a student; the quantiser converts the student into a deployable artifact; the inference engine consumes the artifact and produces translations. The evaluator is a side branch that consumes both student and teacher outputs to compute the ratio that gates deployment.

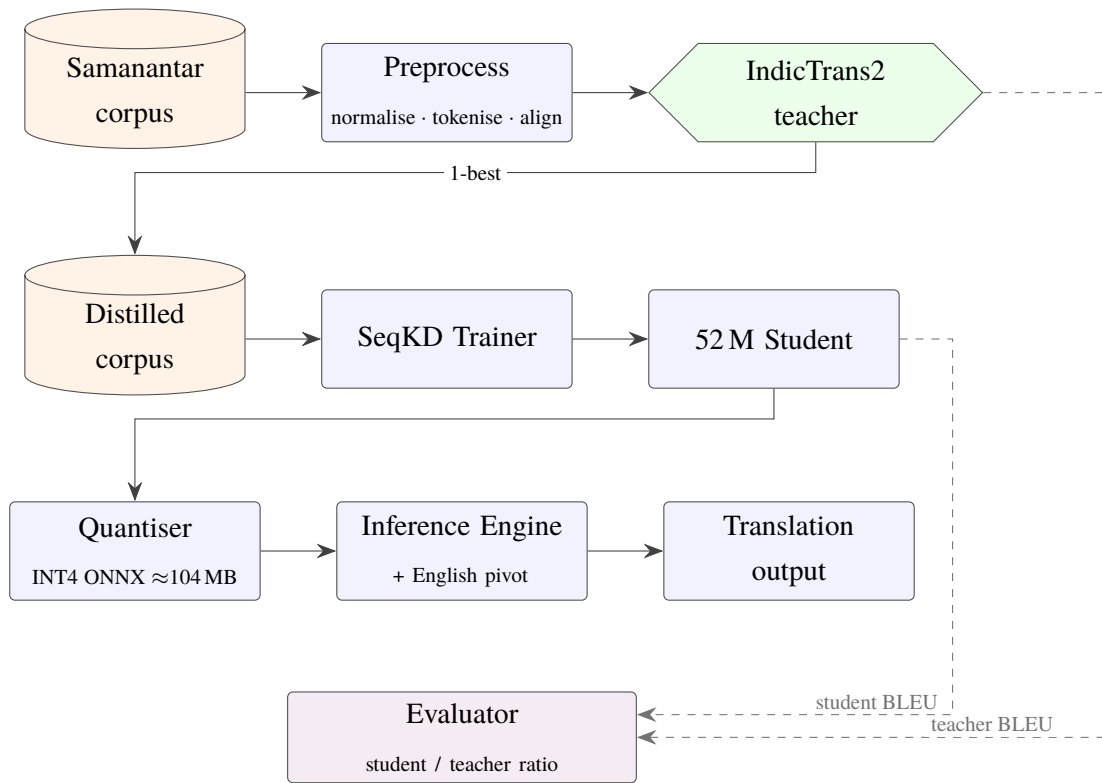


Figure 3.5: Level-1 data flow diagram: corpus to deployed translation

3.2.6 Entity-Relationship Model

SETU does not use a relational database (an offline edge deployment should not require one), but its persistent artifacts have a clear relational structure, recorded in Figure 3.6. Corpora are stored as JSONL, reports as JSON, models as ONNX with an accompanying Sentence-Piece tokenizer, and the language registry as YAML.

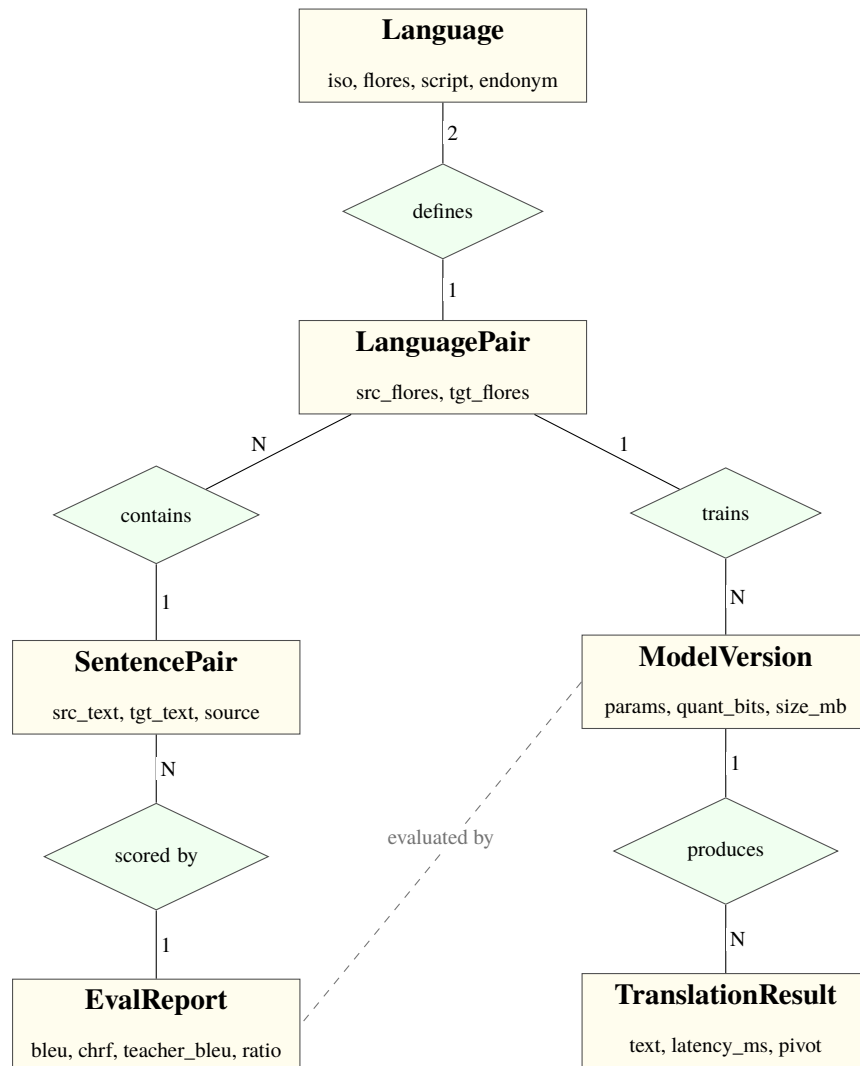


Figure 3.6: Entity-relationship model of SETU's persistent artifacts

3.2.7 Class Diagram

Figure 3.7 gives the principal classes. The design deliberately isolates every dependency on the teacher's implementation inside a single class, `IndicTrans2Teacher`. A unit test scans the source tree and fails the build if any other module imports `transformers`, `torch` or `IndicTransToolkit`. This is what allows the deployed artifact to contain no training dependencies at all.

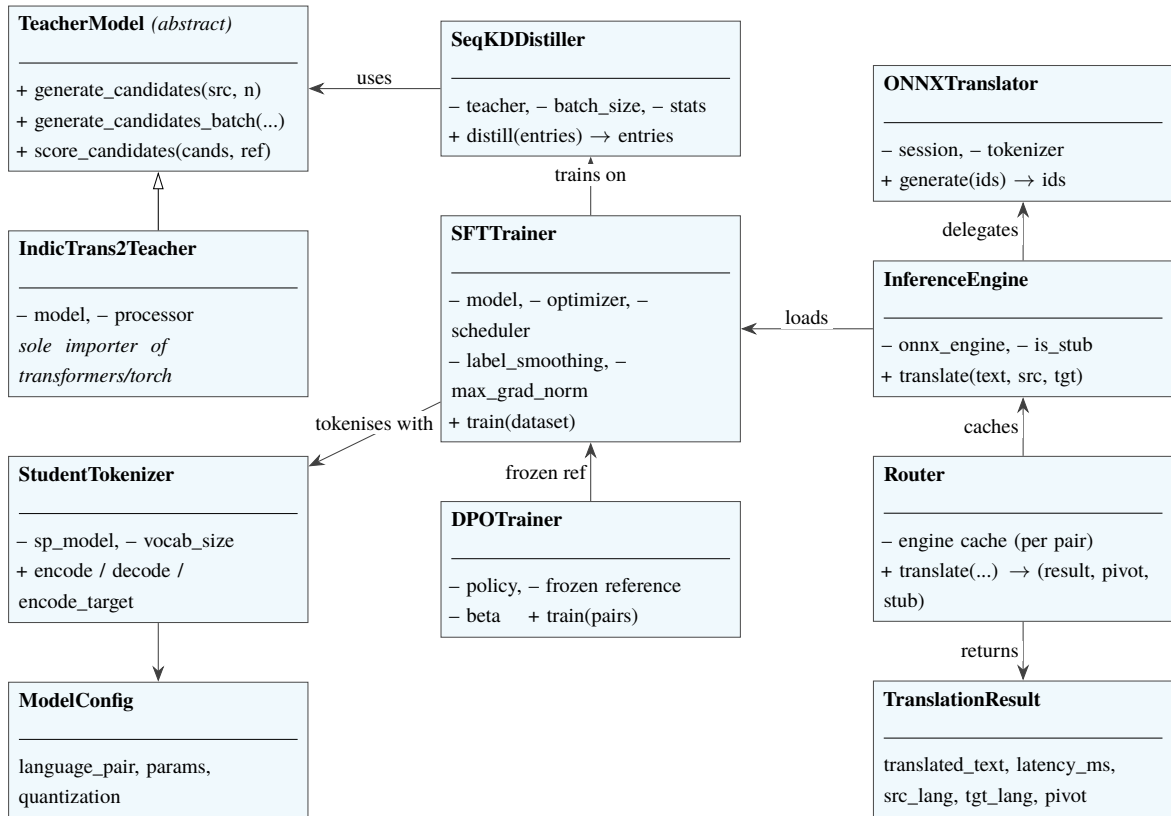


Figure 3.7: Class diagram: training classes (left) are used offline by the ML Engineer; inference classes (right) ship to the device

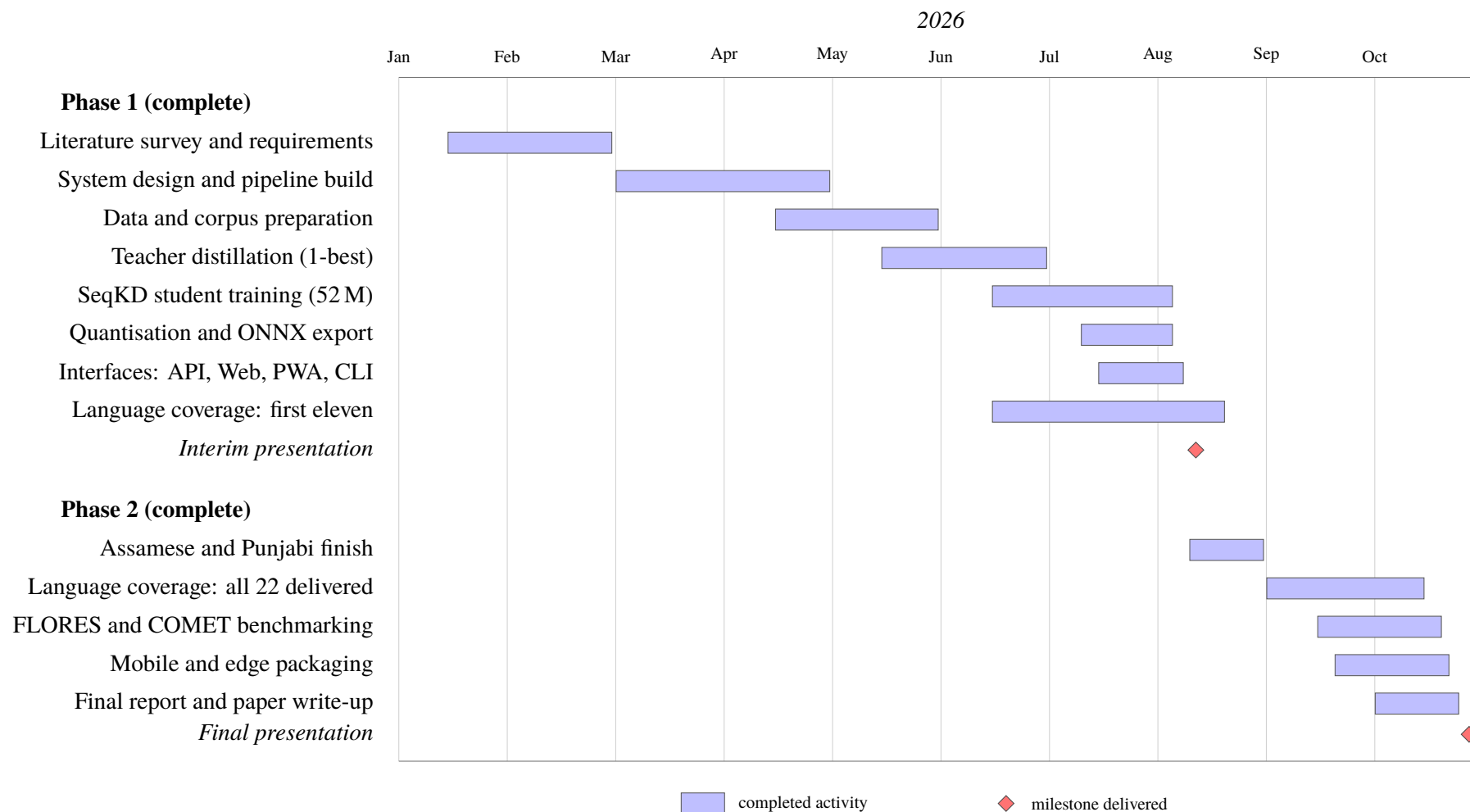


Figure 4.1: Gantt chart: both phases complete, all 22 scheduled languages delivered

the two collapses described in Section 8.4 consumed almost exactly the two weeks that activity G was allotted.

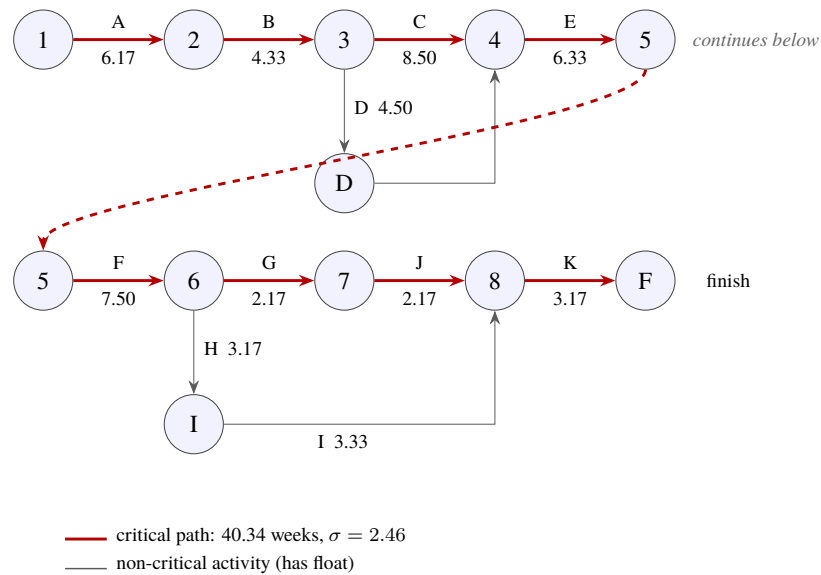


Figure 4.2: PERT network for Phase 1 with the critical path highlighted

4.1.4 Risk Management

Table 4.4 records the risks identified at planning time, together with the mitigation applied and (where the risk materialised) what actually happened. Five of the nine anticipated risks occurred.

Table 4.4: Risk register with outcomes

ID	Risk	Prob.	Impact	Mitigation and outcome
R1	Gated access to the teacher checkpoints and the BPCC corpus	High	High	Fall back to ungated rotary checkpoints and the Samanantar corpus. Occurred. Mitigation applied; the cost is a lower quality ceiling, reported explicitly
R2	No GPU available locally	High	High	Use a cloud GPU (Kaggle, Colab) and later a dedicated A40 VM. Occurred. All final training ran on the A40 VM

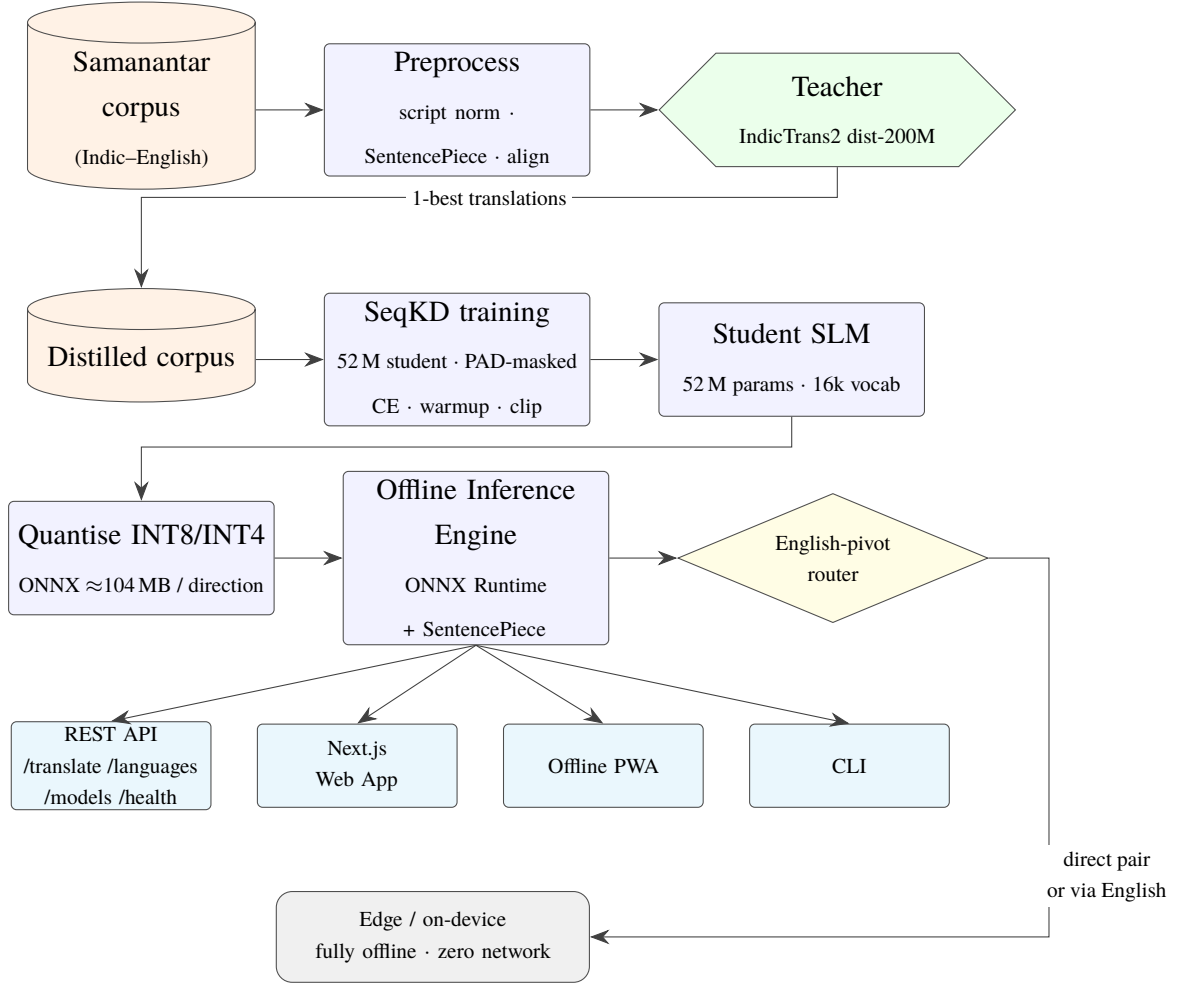


Figure 5.1: System architecture: SeqKD distillation and offline deployment pipeline

The architecture has four layers.

The data layer ingests parallel corpora, normalises and filters them, and stores two corpora per direction: the *processed* corpus, which retains human references and is used only for evaluation, and the *distilled* corpus, whose targets are the teacher’s one-best translations and which is used for training. Keeping both is what allows evaluation to remain on real references regardless of what the student was trained on, the property that makes the comparison in Section 8.1 fair.

The model layer holds the teacher behind an abstract interface and the student behind a trainer. The teacher is used only offline, and only through three methods. The student is a Marian encoder–decoder trained from scratch.

The deployment layer converts a trained student into a device artifact: ONNX export, runtime validation, progressive INT8 and INT4 quantisation, and a per-stage benchmark. Its output is a self-contained directory.

The serving layer loads those directories on demand, caches one engine per direction, routes each request to a direct model or through the English pivot, and presents the

```

1 class TeacherModel:
2     """Abstract teacher. Only IndicTrans2Teacher may import
        transformers/torch."""
3
4     def generate_candidates(self, src_text: str, src_lang: str,
5                             tgt_lang: str, n: int = 4) -> list[str]:
6         """Return n candidate translations, best first."""
7         raise NotImplementedError
8
9     def generate_candidates_batch(self, src_texts: list[str], src_lang: str,
10                                tgt_lang: str, n: int = 4) ->
11                                list[list[str]]:
12        """Batched form -- the only method the SeqKD distiller needs."""
13        raise NotImplementedError
14
15    def score_candidates(self, candidates: list[str],
16                        reference: str) -> list[float]:
17        """Sentence-level chrF of each candidate against the reference."""
18        raise NotImplementedError

```

Listing 6.2: The teacher interface (src/setu/teacher/base.py)

The wall is enforced mechanically. The test below walks the source tree and fails if any module other than the designated backend imports the teacher’s dependencies. Without it the restriction would erode within weeks, and the deployed artifact would silently acquire a PyTorch dependency.

```

1 FORBIDDEN = ("transformers", "torch", "IndicTransToolkit")
2 ALLOWED    = {"src/setu/teacher/indictrans2.py"}
3
4 def test_teacher_wall():
5     """No module outside the backend may import the teacher's dependencies."""
6     for path in Path("src/setu").rglob("*.py"):
7         rel = path.as_posix()
8         if rel in ALLOWED or "/training/" in rel or "/quantize/" in rel:
9             continue                                # training and export legitimately
                                                         use torch
10        source = path.read_text()
11        for name in FORBIDDEN:
12            assert f"import {name}" not in source, f"{rel} imports {name}"

```

Listing 6.3: Enforcing the teacher wall (tests/test_teacher.py)

6.2.2 Corpus Cleaning

The cleaning filters encode domain knowledge about Indic text that a generic text pipeline would get wrong. Two details matter. Zero-width joiners and non-joiners must survive normalisation, because they distinguish valid conjuncts; a naive control-character strip corrupts correct text. And the script-fraction filter is what removes mined pairs in which one side is simply not in the claimed language.

```

1 ZWJ, ZWNJ = "\u200d", "\u200c"    # zero-width joiner / non-joiner
2
3 def normalize(text: str) -> str:
4     """NFC normalise and strip control characters, preserving ZWJ/ZWNJ."""
5     text = unicodedata.normalize("NFC", text)
6     return "".join(
7         ch for ch in text
8         if unicodedata.category(ch) != "Cc" or ch in (ZWJ, ZWNJ, "\n", "\t")
9     )
10
11 def passes_length_ratio(src: str, tgt: str, max_ratio: float = 3.0) -> bool:
12     """Reject misaligned pairs: mined corpora pair unrelated sentences."""
13     if not src or not tgt:
14         return False
15     ratio = max(len(src), len(tgt)) / min(len(src), len(tgt))
16     return ratio <= max_ratio
17
18 def passes_script_fraction(text: str, script: str, minimum: float = 0.5) ->
    bool:
19     """At least `minimum` of the letters must belong to the expected
        script."""
20     letters = [ch for ch in text if ch.isalpha()]
21     if not letters:
22         return False
23     in_script = sum(1 for ch in letters if script_of(ch) == script)
24     return in_script / len(letters) >= minimum

```

Listing 6.4: Corpus filters (src/setu/corpus/filters.py)

6.2.3 The SeqKD Distiller

The distiller is short, which is appropriate: sequence-level distillation is a simple idea, and the project’s contribution is the evidence that it is the right one, not the code that implements it.

```

1 class SeqKDDistiller:
2     """Replace each entry's human target with the teacher's 1-best

```

```

translation."""
3
4     def __init__(self, teacher: TeacherModel, batch_size: int = 32):
5         self.teacher = teacher
6         self.batch_size = batch_size
7         self.stats = {"entries": 0, "empty": 0}
8
9     def distill(self, entries: list[CorpusEntry]) -> list[CorpusEntry]:
10        out: list[CorpusEntry] = []
11        for start in range(0, len(entries), self.batch_size):
12            chunk = entries[start:start + self.batch_size]
13            cands = self.teacher.generate_candidates_batch(
14                [e.src_text for e in chunk],
15                chunk[0].src_lang, chunk[0].tgt_lang, n=1,
16            )
17            for e, c in zip(chunk, cands):
18                if c and c[0].strip():
19                    tgt = c[0]                                # the teacher's target
19                    replaces the human one
20                else:
21                    tgt = e.tgt_text                          # safe fallback, never a
21                    blank target
22                    self.stats["empty"] += 1
23                    out.append(CorpusEntry(e.src_lang, e.tgt_lang,
24                                            e.src_text, tgt, "seqkd"))
25                    self.stats["entries"] += len(chunk)
26        return out

```

Listing 6.5: Sequence-level distillation (src/setu/distill/distiller.py)

The source field is set to "seqkd" so that a distilled corpus can never be mistaken for a reference corpus downstream. The evaluation code asserts that it is scoring against entries whose source is *not* seqkd, which is the mechanical guarantee behind the fairness of Section 8.1.

6.2.4 The Training Loop

This is the most-debugged code in the project. Every line commented below corresponds to a failure that was observed and diagnosed; the run history is in Section 8.3.

```

1     def train(self, dataset) -> dict:
2         total_steps = len(dataloader) * self.epochs
3         # Warmup was previously read from config and silently ignored; a constant
3         LR
4         # underfits a from-scratch transformer. Cap warmup at 10% of the run.

```

```

5     warmup = min(self.config.get("warmup_steps", 0), max(1, total_steps //
6         10))
7
8     for epoch in range(self.epochs):
9         for batch in dataloader:
10             outputs = self.model(input_ids=batch["input_ids"],
11                                   attention_mask=batch["attention_mask"],
12                                   labels=batch["labels"]) # labels use -100
13                                                         at PAD
14             if self.label_smoothing > 0:
15                 loss = F.cross_entropy(
16                     outputs.logits.view(-1, outputs.logits.size(-1)),
17                     batch["labels"].view(-1),
18                     ignore_index=-100, # never score padding
19                     label_smoothing=self.label_smoothing,
20                 )
21             else:
22                 loss = outputs.loss
23                 loss.backward()
24                 # The single most important stabiliser: without clipping, a
25                 # 200k-sentence
26                 # run collapsed to constant output while 120k on the same code
27                 # succeeded.
28                 torch.nn.utils.clip_grad_norm_(self.model.parameters(),
29                                                  self.max_grad_norm)
30                 self.optimizer.step()
31                 scheduler.step()
32                 self.optimizer.zero_grad()

```

Listing 6.6: Supervised training step with all four stabilisers (src/setu/training/sft.py)

The dataset builds the labels, and the single most consequential line in the entire training path is the padding mask:

```

1 def encode_target(self, text: str) -> list[int]:
2     """Labels are [tokens, EOS] with NO leading BOS: HuggingFace already
3     prepends
4     the decoder start token, so adding one wastes a decode step and creates a
5     train/generate mismatch that produced empty output in an early run."""
6     ids = self.sp.encode(text)[: self.max_seq_len - 1]
7     return ids + [self.eos_id]
8
9 def collate(self, batch: list[dict]) -> dict:

```

```

9     labels = pad_sequence([b["labels"] for b in batch], batch_first=True,
10                           padding_value=PAD_ID)
11     # -100, not PAD: HuggingFace's loss uses ignore_index=-100. Padding with
12     # PAD
13     # gave the model free credit for predicting PAD, producing a fake-low
14     # loss of
15     # 0.274 and a decoder that ignored the source entirely.
16     labels[labels == PAD_ID] = -100
17     return {"input_ids": ..., "attention_mask": ..., "labels": labels}

```

Listing 6.7: Label construction (src/setu/training/dataset.py)

6.2.5 The Inference Router

The router is the whole of SETU’s multilingual coverage story. It is also the one place where routing logic exists, which is what prevents the interfaces from diverging.

```

1  @lru_cache(maxsize=None)
2  def engine_for_pair(pair: str) -> InferenceEngine:
3      """One cached engine per pair; a pair with no model on disk stays a
4      stub."""
5
6      return InferenceEngine(ModelConfig(language_pair=pair))
7
8  def translate(text, src_lang, tgt_lang) -> tuple[TranslationResult, str |
9  None, bool]:
10     """Returns (result, pivot, stub). Raises ValueError on unknown or equal
11     codes."""
12
13     src, tgt = resolve_language(src_lang), resolve_language(tgt_lang)
14     if src["iso"] == tgt["iso"]:
15         raise ValueError(f"Source and target language are both
16         {src['iso']!r}")
17
18     direct = f"{src['flores']}-{tgt['flores']}"
19     if _available(direct):
20         return engine_for_pair(direct).translate(text, src["iso"],
21         tgt["iso"]), None, False
22
23     # No direct model: chain source -> English -> target if both halves exist.
24     eng = load_languages()["en"]["flores"] # eng_Latn
25     if src["iso"] != "en" and tgt["iso"] != "en":
26         hop1, hop2 = f"{src['flores']}-{eng}", f"{eng}-{tgt['flores']}"
27         if _available(hop1) and _available(hop2):
28             r1 = engine_for_pair(hop1).translate(text, src["iso"], "en")
29             r2 = engine_for_pair(hop2).translate(r1.translated_text, "en",
30             tgt["iso"])

```

```

23         return (TranslationResult(
24             translated_text=r2.translated_text,
25             src_lang=src["iso"], tgt_lang=tgt["iso"],
26             latency_ms=(r1.latency_ms or 0.0) + (r2.latency_ms or
27                 0.0)),
28             "en", False)
29     # Nothing can serve this pair: pass through, but say so.
30     return engine_for_pair(direct).translate(text, src["iso"], tgt["iso"]),
31         None, True

```

Listing 6.8: English-pivot routing (src/setu/inference/router.py)

6.2.6 Offline Verification

The offline guarantee is a claim about the absence of behaviour, which cannot be established by inspection alone. It is therefore tested by removing the capability entirely: all socket construction is patched to raise, and translation is then required to succeed.

```

1  def assert_offline():
2      """Disable every socket so any network call raises instead of
3         succeeding."""
4      def _blocked(*args, **kwargs):
5          raise RuntimeError("network access attempted during offline
6              inference")
7      socket.socket = _blocked
8      socket.create_connection = _blocked
9
10     def test_onnx_engine_translates_offline(tmp_models):
11         assert_offline() # no sockets from here on
12         engine = InferenceEngine(ModelConfig(language_pair="hin_Deva-eng_Latn"))
13         result = engine.translate("bhaarat ek vishaal desh hai.", "hi", "en")
14         assert result.translated_text # non-empty
15         assert not engine.is_stub # a real model
16         answered

```

Listing 6.9: Proving offline operation (tests/test_quantize.py)

6.2.7 The REST Service

```

1  @app.post("/translate", response_model=TranslateResponse)
2  def translate(req: TranslateRequest) -> TranslateResponse:
3      try:
4          result, pivot, stub = router.translate(req.text, req.source_lang,
5              req.target_lang)

```

```

5     except ValueError as exc:                                # unknown code, or src == tgt
6         raise HTTPException(status_code=400, detail=str(exc))
7     return TranslateResponse(
8         translated_text=result.translated_text,
9         source_lang=req.source_lang, target_lang=req.target_lang,
10        latency_ms=result.latency_ms,
11        pivot=pivot,      # "en" when routed through English, else null
12        stub=stub,        # true means passthrough: never mistake it for a
                           translation
13    )
14
15 @app.get("/models")
16 def models() -> dict:
17     """Discover every model directory on disk with its variant and size."""
18     return {"count": len(_scan_models()), "models": _scan_models()}

```

Listing 6.10: Translation endpoint (interfaces/rest/app.py)

6.2.8 Code Efficiency and Optimisation

Efficiency in SETU means fitting three budgets: training memory, artifact size and inference latency. Table 6.1 records the optimisations applied and their measured effect.

Table 6.1: Optimisations applied and their measured effect

Optimisation	Problem addressed	Measured effect
Progressive INT8 then INT4 quantisation	410.9 MB FP32 artifact is over twice the 200 MB budget	410.9 → 104.6 → 103.9 MB, a 4.0× compression, with negligible quality loss
Greedy decoding at deployment	Beam search costs roughly 4× the latency	p90 latency falls to 141–350 ms for most directions; beam search retained for offline evaluation only
Corpus streaming	The full Samanantar corpus is tens of gigabytes	Only the requested slice is ever materialised
Greedy teacher decoding for distillation	Beam-5 distillation of 500,000 sentences does not fit a session	4–5× faster; target quality unaffected
Per-pair engine cache (lru_cache)	Reloading a 104 MB ONNX session per request	Each direction loads from disk at most once

7.1.2 Unit Testing

Unit tests exercise one module in isolation. The tables below give the cases for the components whose failure modes matter most, each with at least one valid and two invalid inputs.

7.1.2.1 Translation Engine

Table 7.2: Unit test UT-1 to UT-5: translation engine, Hindi→English INT4 ONNX student

#	Test data (input)	Expected result	Actual result	Verdict
UT-1	src=hi, tgt=en, भारत एक विशाल देश है।	Fluent English translation	<i>“India is a vast country.”</i>	Pass
UT-2	src=hi, tgt=hi (identical languages)	Rejected with HTTP 400 / ValueError	HTTP 400: <i>source and target are both ‘hi’</i>	Pass
UT-3	src=xx, tgt=en (unknown language code)	Rejected: unknown language code	ValueError raised; request refused	Pass
UT-4	Empty string input	Empty output, no exception	" " returned; no exception	Pass
UT-5	Sentence exceeding max_seq_len	Truncated and translated, not crashed	Truncated to the limit; translation returned	Pass

7.1.2.2 Corpus Pipeline

Table 7.3: Unit tests UT-6 to UT-10: corpus cleaning filters

#	Test data (input)	Expected result	Actual result	Verdict
UT-6	Well-formed Hindi–English pair	Retained	Retained	Pass
UT-7	Pair with length ratio 5:1	Rejected (ratio > 3.0)	Rejected, counted as <code>length_ratio</code>	Pass
UT-8	Devanagari source paired with a Tamil “English” side	Rejected (script fraction below 0.5)	Rejected, counted as <code>wrong_script</code>	Pass
UT-9	Text containing ZWJ and ZWNJ	Both preserved through normalisation	Both preserved; NFC applied	Pass
UT-10	Duplicate of an earlier pair	Removed	Removed, counted as <code>duplicate</code>	Pass

7.1.2.3 Training Mechanics

These are the property tests. Each asserts something that cannot be verified by reading the output of a training run.

Table 7.4: Unit tests UT-11 to UT-15: training mechanics

#	Property under test	Expected result	Actual result	Verdict
UT-11	DPO reference model is frozen	No parameter of the reference receives a gradient	All reference gradients None	Pass
UT-12	Training loss decreases over a short run	Final loss below initial	Loss decreased monotonically	Pass
UT-13	Padding is excluded from the loss	Label tensor contains -100 at every pad position	Verified for every batch	Pass
UT-14	EWC penalty is zero at the snapshot	Penalty = 0 at θ^* , > 0 away from it	0.0 at snapshot; grows with distance	Pass
UT-15	AIO-KD samples every direction	Coverage report lists all configured directions	All directions sampled	Pass

7.1.3 Integration Testing

Integration tests exercise two or more modules together. The most important is the English pivot, because it is the only path in the system in which the output of one model becomes the input of another.

7.1.3.1 Inference Engine and Pivot Router

Table 7.5: Integration tests IT-1 to IT-4: English-pivot chaining for Indic-to-Indic pairs

#	Test data (input)	Expected result	Actual result	Verdict
IT-1	src=hi, tgt=bn, भारत एक विशाल देश है। (no direct model)	Route via English; return Bengali	ভারত একটি বিশাল দেশ। (pivot="en", ≈57 ms)	Pass
IT-2	src=ta, tgt=te, இந்தியா ஒரு பெரிய நாடு.	Route via English; return Telugu	భారతదేశం ఒక పెద్ద దేశం. (pivot="en")	Pass
IT-3	src=as, tgt=bn, ভারত এখন ডাঙৰ দেশ।	Route via English; return Bengali	ভারত একটি বড় দেশ। (pivot="en")	Pass
IT-4	src=mr, tgt=kn	Route via English; return Kannada	Correct Kannada returned, ≈50–65 ms	Pass

7.1.3.2 REST API and Inference Engine

Table 7.6: Integration tests IT-5 to IT-9: REST service against the routing layer

#	Request	Expected result	Actual result	Verdict
IT-5	GET /models	All installed pairs with variant and size	count: 22, each int4, 103.9 MB	Pass
IT-6	GET /languages	22 scheduled languages plus English	23 entries with ISO, FLORES, script, endonym	Pass
IT-7	POST /translate with ta→en	Real translation, stub:false	Correct English, ≈40–60 ms, stub:false	Pass
IT-8	POST /translate with an unknown code	HTTP 400 with a descriptive message	HTTP 400 returned	Pass
IT-9	CORS preflight from the browser client	Preflight accepted	OPTIONS returns the expected headers	Pass

7.1.3.3 Full Pipeline

Table 7.7: Integration tests IT-10 to IT-13: end-to-end pipeline

#	Scenario	Expected result	Actual result	Verdict
IT-10	Corpus → distil → train → evaluate on a small slice	Runs end to end; a training report is written	Report written with BLEU, chrF and ratio	Pass
IT-11	Train → export → quantise → translate offline	Quantised model translates with sockets disabled	Translation returned; no network call	Pass
IT-12	Same pipeline on a non-default language pair	Pair-agnostic: succeeds without code change	Tamil→English and reverse both succeeded	Pass
IT-13	CLI and REST for the same request	Identical output from both interfaces	Identical, since both call the same router	Pass

IT-13 deserves a note, because it was added in response to a genuine defect. The CLI originally constructed an `InferenceEngine` with the language pair from configuration rather than from the request, so `--src` as loaded the Hindi model and translated with it. The output looked like a translation and was wrong. The fix, routing both interfaces through `setu.inference.router`, is exactly what this test now enforces.

7.1.4 System and Non-Functional Testing

Table 7.8: System tests ST-1 to ST-6: non-functional targets

#	Attribute	Target	Measured	Verdict
ST-1	Artifact size	≤ 200 MB per direction	103.9 MB for all 22 directions	Pass
ST-2	Inference latency	< 500 ms per sentence	Mean 105–202 ms; p90 141–541 ms	Pass (one p90 outlier)
ST-3	Offline operation	No network call at inference	All sockets disabled; translation succeeded	Pass
ST-4	Quality ratio	≥ 0.80 of teacher BLEU	14 of 22 directions; mean 0.838	Partial
ST-5	Model health across all directions	Every model produces non-degenerate output	22 of 22 healthy; no empty, passthrough or collapsed output	Pass
ST-6	Coverage	All pairs among supported languages reachable	22 direct plus English-pivoted routes	Pass

ST-5 was implemented as a dedicated script, `scripts/translation_suite.py`, which loads each model in turn, translates a real source sentence in that model’s own direction, and flags three specific pathologies: empty output, output identical to the input (a passthrough masquerading as a translation), and collapsed output (the same string returned for different inputs). It is the test that confirmed the two recovered models were genuinely repaired rather than merely scoring better.

7.1.5 Defects Found by Testing

Table 7.9 lists the defects the suite caught, with the test that caught each. They are reported because the class of defect is instructive: most were silent, producing plausible output rather than an error.

Table 8.6: Full scorecard: 22 directions, 500-sentence Samanantar development slice, INT4 ONNX students

#	Direction	BLEU	chrF	Teacher	Ratio	≥ 0.80	Latency
1	Marathi $\rightarrow$ English	16.4	43.8	17.1	0.955	Pass	131 ms
2	English $\rightarrow$ Kannada	10.0	43.0	11.0	0.915	Pass	105 ms
3	English $\rightarrow$ Punjabi	15.8	42.3	17.5	0.904	Pass	175 ms
4	English $\rightarrow$ Marathi	12.5	40.9	13.8	0.901	Pass	111 ms
5	Odia $\rightarrow$ English	18.9	46.5	21.0	0.898	Pass	133 ms
6	Malayalam $\rightarrow$ English	17.1	44.6	19.1	0.898	Pass	128 ms
7	English $\rightarrow$ Odia	6.9	40.8	7.7	0.895	Pass	128 ms
8	Punjabi $\rightarrow$ English	21.1	47.6	24.0	0.877	Pass	158 ms
9	English $\rightarrow$ Hindi	20.3	46.6	23.3	0.871	Pass	202 ms
10	English $\rightarrow$ Malayalam	5.2	38.8	6.1	0.863	Pass	113 ms
11	English $\rightarrow$ Gujarati	12.5	41.1	14.6	0.855	Pass	114 ms
12	English $\rightarrow$ Bengali	11.1	41.5	13.1	0.848	Pass	130 ms
13	Telugu $\rightarrow$ English	18.9	46.5	23.1	0.817	Pass	129 ms
14	Hindi $\rightarrow$ English	21.8	49.5	27.1	0.806	Pass	199 ms
15	Gujarati $\rightarrow$ English	18.8	46.2	23.9	0.787	Near	130 ms
16	English $\rightarrow$ Telugu	11.0	42.8	14.0	0.780	Near	113 ms
17	English $\rightarrow$ Assamese	6.7	33.7	8.7	0.764	Near	155 ms
18	Kannada $\rightarrow$ English	16.3	45.6	21.4	0.763	Near	132 ms
19	Tamil $\rightarrow$ English	16.1	41.0	21.1	0.761	Near	155 ms
20	Assamese $\rightarrow$ English	13.1	33.5	17.3	0.757	Near	154 ms
21	Bengali $\rightarrow$ English	19.1	45.2	25.2	0.756	Near	150 ms
22	English $\rightarrow$ Tamil	8.0	43.0	10.6	0.755	Near	135 ms

several, so n-gram overlap is structurally low even when the translation is correct. This is exactly why the project gates on the *ratio* rather than absolute BLEU, and why COMET scoring is recommended for Malayalam and Tamil: several near-misses may already pass under a metric that is not overlap-based.

8.5.3 Deployment Metrics: Size and Latency

Table 8.9: Deployment metrics per direction (INT4 ONNX, VM host CPU)

Direction	FP32	INT8	INT4	≤ 200	p50	p90	< 500
Marathi $\rightarrow$ English	411	104.6	103.9	Yes	95 ms	147 ms	Yes
English $\rightarrow$ Kannada	411	104.6	103.9	Yes	62 ms	141 ms	Yes
English $\rightarrow$ Marathi	411	104.6	103.9	Yes	78 ms	156 ms	Yes
Odia $\rightarrow$ English	411	104.6	103.9	Yes	78 ms	147 ms	Yes
Malayalam $\rightarrow$ English	411	104.6	103.9	Yes	86 ms	213 ms	Yes
English $\rightarrow$ Odia	411	104.6	103.9	Yes	201 ms	350 ms	Yes
English $\rightarrow$ Hindi	411	104.6	103.9	Yes	124 ms	300 ms	Yes
English $\rightarrow$ Malayalam	411	104.6	103.9	Yes	85 ms	173 ms	Yes
English $\rightarrow$ Gujarati	411	104.6	103.9	Yes	74 ms	148 ms	Yes
English $\rightarrow$ Bengali	411	104.6	103.9	Yes	179 ms	336 ms	Yes
Telugu $\rightarrow$ English	411	104.6	103.9	Yes	86 ms	204 ms	Yes
Hindi $\rightarrow$ English	411	104.6	103.9	Yes	134 ms	335 ms	Yes
Gujarati $\rightarrow$ English	411	104.6	103.9	Yes	82 ms	165 ms	Yes
English $\rightarrow$ Telugu	411	104.6	103.9	Yes	72 ms	200 ms	Yes
Kannada $\rightarrow$ English	411	104.6	103.9	Yes	136 ms	281 ms	Yes
Tamil $\rightarrow$ English	411	104.6	103.9	Yes	93 ms	185 ms	Yes
Bengali $\rightarrow$ English	411	104.6	103.9	Yes	88 ms	190 ms	Yes
English $\rightarrow$ Tamil	411	104.6	103.9	Yes	214 ms	541 ms	Near

Every model compresses by a factor of 4.0, from 410.9 MB to 103.9 MB, which is 48 % below the 200 MB budget. Median latency is comfortably inside the 500 ms budget

for every direction. The single exception is English→Tamil, whose p90 of 541 ms exceeds the budget; its median of 214 ms does not. This was measured on a shared VM host under contention, and Tamil’s longer output sequences make it the most sensitive direction to host load. It is recorded as a near-miss rather than discounted.

8.5.4 Definition-of-Done Scorecard

Table 8.10: Final scorecard against the four project targets

Target	Metric	Result	Threshold	Status
Quality	Student / teacher BLEU ratio	14 of 22 directions; mean 0.838, median 0.855, best 0.955	≥ 0.80	Partial
Latency	Milliseconds per sentence, quantised	Mean 105–202 ms; p90 141–541 ms	< 500 ms	Pass
Size	Quantised artifact, INT4	103.9 MB, every direction	≤ 200 MB	Pass
Offline	Inference with networking disabled	Succeeds for every direction	Zero network calls	Pass

Three of the four targets pass without exception across all 22 directions. Quality is met on 14 of 22. On the primary development pair, Hindi→English, all four targets are met and the definition of done is complete.

8.6 Qualitative Analysis of Translation Quality

Metrics measure aggregate overlap; they do not show whether output is usable. This section examines what the models actually produce.